Task 2 - Datasheet Literacy: TB6612FNG Dual DC Motor Driver

Features

- Power supply voltage ： VM=15V (Max.)
- Output current ： Iout=1.2A(ave) / 3.2A (peak)
- Output low ON resistor ： 0.5Ω (upper+lower Typ. @VM≧ 5V)
- Standby (Power save) system
- CW/CCW/short brake/stop function modes
- Built-in thermal shutdown circuit and low voltage detecting circuit
- Small faced package (SSOP24 : 0.65mm Lead pitch)
- Response to Pb free packaging

1. Datasheet Parameters Constraining Circuit Design

Based on the TB6612FNG specifications, the following three parameters directly constrained the BOM, wiring, and software decisions:

- **Logic Supply Voltage Range (V_{cc}):** The datasheet specifies that the logic supply voltage (V_{cc}) operates safely between 2.7V and 5.5V.
    - *Design Influence:* This perfectly brackets the ESP32's native 3.3V logic level. Consequently, the wiring design routes the ESP32's 3V3 output pin directly to the TB6612FNG's V_{cc} pin. This ensures the PWM and directional logic signals from the ESP32 (which operate at 3.3V) are recognized cleanly without requiring any external logic level-shifter modules in the BOM.
- **Continuous Output Current (I_{out}):** The datasheet rates the driver for a maximum continuous output current of 1.2A per channel.
    - *Design Influence:* Since this is a 2WD maze solver,one motor will be wired in Channel A and B each. This specification heavily constrained the BOM: the chosen N20 gear motors must have an active load current of strictly less than 1.2A each, guaranteeing that the combined current draw per side never exceeds the driver's 1.2A continuous thermal limit.
- **Maximum PWM Switching Frequency (F_{PWM})**: The operating range table dictates that the PWM frequency supplied to the driver must not exceed 100 kHz.
    - *Design Influence:* This parameter directly constraints the ESP32's software configuration. The ESP32's internal PWM timers (ledcSetup) will be configured to output a frequency of 20 kHz. This is well below the driver's 100 kHz maximum limit, but fast enough to push the switching noise above the range of human hearing, ensuring the robot drives smoothly and silently.

2. Destructive Conditions & Design Mitigation

- **Absolute Maximum Motor Supply Voltage (V_M):** The "Absolute Maximum Ratings" table explicitly states that the motor power supply (V_M) must never exceed 15V. Violating this rating will break down the internal MOSFETs and permanently destroy the integrated circuit.
    - *How the design avoids it:* The BOM specifies a 3S LiPo battery as the primary power source for the motors. A fully charged 3S LiPo battery peaks at exactly 12.6V. Wiring this battery directly to the V_M pin provides maximum torque

while guaranteeing a permanent 2.4V safety buffer below the chip's absolute destruction threshold, ensuring stable operation even when fully charged.

Task 3 — PID Control for Line Following

Error convention: **error > 0 means the line is to the right of centre; correction > 0 means turn right** (left wheel faster, right wheel slower).

1. The role of P, I and D

Term	Looks at	Effect on the robot
P (proportional)	the current error	Steers in proportion to how far the line is off-centre. Too small: sluggish, loses the line in curves. Too large: the robot hunts left and right.
D (derivative)	how fast the error is changing	Damps the motion. When the error is shrinking quickly, D reduces the correction so the robot doesn't overshoot the line.
I (integral)	the error accumulated over time	Removes a small constant offset (for example from mismatched motors) by pushing harder the longer the offset lasts.

Why D is needed even though P points the right way. A line follower steers by changing its *heading*, not by sliding sideways. When the robot reaches the centre, the error is 0, so P outputs "go straight", but the robot is still pointed at an angle, so it drifts across the line. P cannot see the heading. D can detect it indirectly, because a fast-shrinking error means the robot is angled toward the line, and it straightens the robot before it arrives.

2. Chosen configuration: PD (Ki = 0)

- **P** is needed for the basic correction.
- **D** is needed because the robot has to run fast and P alone overshoots (explained above).
- **I is left out.** The maze consists of sharp 90° turns, where the error stays large for a while. I would accumulate all of that error and keep pushing after the turn is finished (integral windup), causing overshoot. I also cannot tell a motor bias apart from a turn.
- A constant motor mismatch is instead handled by a **trim offset** (`leftTrim`) on the base speed, found by driving straight on a flat surface. If testing still showed a steady offset on straights, I would add a very small Ki with the integral clamped (the code already has this clamp, with Ki set to 0).

3. From sensors to motor commands

1. **Calibrate** at startup: record each sensor's min and max while sweeping over line and floor, then rescale every reading to 0–1000 (`(raw - min) / (max - min) × 1000`), flipping it if needed so that line = high.

2. **Centroid:** `position = Σ(weight × reading) / Σ(reading)` with weights `[-2, -1, 0, 1, 2]`, giving a value from −2 to +2.
3. **Error:** `error = position - 0` (the target is the line at the centre).
4. **PD correction:** `correction = Kp × error + Kd × (error - lastError) / dt` (run in a fixed-period loop so `dt` is known).
5. **Differential speeds:** `left = base + correction`, `right = base - correction`.
6. **Clamp** both values to the PWM range before writing to the motor driver.

Worked example: (base = 150, Kp = 40, readings `[0, 0, 200, 800, 0]`): position = 800 / 1000 = 0.8, so error = 0.8 and correction = 40 × 0.8 = 32. Left = 182 and right = 118: the left wheel is faster, so the robot turns right, toward the line. With Kd = 0.5 and a next-step error of 0.5 after dt = 0.01 s: D = 0.5 × (0.5 − 0.8) / 0.01 = −15, so the correction is 40 × 0.5 − 15 = 5 instead of 20. D has reduced the push because the error is closing fast.

4. Oscillation and lost line

Oscillation (robot wobbling across the line)

1. Lower `baseSpeed` so there is less momentum.
2. Tune Kp first with Kd = 0: raise it until the robot just starts to wobble, then reduce it by roughly a third to a half.
3. Raise Kd until the wobble dies out. Too much Kd makes the robot jittery, because D amplifies sensor noise; the code low-pass filters the derivative to limit this.
4. Keep Ki at 0.

Lost line (all sensors read low)

- The code treats a total calibrated signal of exactly 0 (all five sensors read 0) as "line lost". This is checked before dividing, so there is no divide-by-zero.
- **Recovery:** pivot on the spot toward the side where the line was last seen (the sign of the last valid position). A line lost at a sharp turn is almost always on the side of the turn.
- **Safe stop:** if the line is not found within a timeout, stop both motors and stay stopped.
- When the line is found again, the PID's stored derivative and integral are reset so stale values don't cause a jerk.

Task 4: BOM notes

Major Component Choice Justification:

Motor Driver: TB6612FNG vs. L298N For a high-speed line maze solver, the **TB6612FNG** was selected over the traditional **L298N** motor driver.

- **Voltage Drop & Efficiency:** The L298N uses older, inefficient bipolar junction transistors (BJTs) which cause a massive voltage drop of roughly 2V to 3V. If supplied with 11.1V, the motors would only receive ~8.5V, wasting the rest as heat.

The TB6612FNG uses modern MOSFETs with a negligible voltage drop, delivering nearly the full 11.1V to the N20 motors to achieve their maximum 600 RPM potential.

Task 5: Power Budget Assumptions and Runtime Analysis

1. Runtime Calculation & Stated Assumptions The system's power source is a 3S Lithium Polymer (LiPo) battery with a capacity of 850mAh and a nominal voltage of 11.1V. The total nominal system current is estimated at 350mA based on the following hardware assumptions:

- **Motor Nominal Current:** The datasheet for the GA12-N20 600 RPM 12V motors omits the free-running current but lists a stall current of 750mA. Using standard micro-metal gearmotor characteristics, nominal current is estimated at approximately 10% to 15% of stall, yielding a safe budget estimate of 100mA per motor (200mA total for the 2WD chassis).
- **Logic & Sensor Current:** The ESP32 microcontroller operates with Wi-Fi and Bluetooth disabled to conserve power. Running a standard PID control loop and polling the 3.3V TCRT5000 IR sensor array, the logic branch is estimated to draw a constant 150mA.

Runtime Calculation:

- Formula: Battery Capacity (mAh) ÷ Total Nominal Current Draw (mA)
- Calculation: 850 mAh ÷ 350 mA = 2.428 hours
- **Estimated Runtime:** Under continuous typical load, the robot will operate for approximately **2.43 hours** (2 hours and 26 minutes) per charge.

2. Cell Chemistry Justification (LiPo vs. Li-ion) We selected Lithium Polymer (LiPo) over Lithium-Ion (Li-ion) to maximize the 2WD solver's agility and electrical stability:

- **Weight:** Li-ion uses heavy metal cylinders (e.g., 18650s), which increases chassis momentum and causes skidding on 90-degree turns. Our LiPo pouch cell weighs only 75g, keeping the robot light and nimble.
- **C-Rating:** Standard Li-ion cells have lower discharge rates (5C–10C). Our LiPo's 30C rating easily delivers up to 25.5 Amps, completely preventing voltage sag and ESP32 brownouts during a 1.75A worst-case motor stall.
- **Energy Density & Cost:** While Li-ion offers slightly better volumetric energy density and a lower cost per mAh, LiPo's superior power-to-weight ratio is mandatory for a timed maze competition where speed and cornering outweigh maximum capacity and budget.

Here is the complete, professional troubleshooting analysis for Task 6. You can copy and paste this directly into your `docs/` folder alongside your wiring diagram notes.

Task 6: Electronics & Hardware — Troubleshooting Analysis

Selected Fault: Controller Resetting / Brownout When Motors Engage

- **Failure Symptom:** The ESP32 microcontroller spontaneously reboots, disconnects, or loses its operational state whenever the 600 RPM N20 motors abruptly start up, reverse direction, or experience a load spike.
- **Probable Root Causes:**
    1. *Transient Voltage Sag:* When DC gearmotors start or stall, they draw a high instantaneous current spike (up to 1.75A worst-case total). If the battery's internal resistance causes a temporary voltage drop, or if the input voltage falls below the dropout threshold of the MP1584 buck converter, the 5V logic rail dips, triggering an immediate brownout reset on the ESP32.
    2. *Electromagnetic Interference (EMI) / Inductive Kickback:* Brushed DC motors generate severe high-frequency electrical noise and voltage spikes through brush sparking. Without proper decoupling, this noise couples into adjacent signal lines or power planes, crashing the ESP32's microcontroller clock or Wi-Fi stack.
- **First Diagnostic Check to Perform:**

    1.Connect a digital multimeter (set to DC voltage) or an oscilloscope across the ESP32's VIN and GND pins. Trigger a sudden motor start command and observe whether the voltage drops below ~4.5V during the initial acceleration phase.

    2.Alternatively, check if disabling the motors in software (running only the logic loop) completely stops the resets, isolating the issue strictly to the power bus.
- **How to Confirm and Fix:**
    1. **Hardware Mitigations:**
        - *Bulk Capacitance:* Solder a **10µF to 100µF electrolytic capacitor** in parallel across the TB6612FNG motor driver's VMOT and GND power pins to act as a local energy reservoir that buffers instantaneous current demands.
        - *EMI Suppression:* Solder three small **0.1µF ceramic capacitors** directly onto each N20 motor terminal can (one across the two terminals, and one from each terminal to the metal motor housing) to shunt high-frequency noise to the chassis ground.
    2. **Software Mitigation (Soft-Start Implementation):**

Modify the motor control function to implement a **PWM ramp-up loop** (e.g., incrementing motor speed from 0 to target speed in small steps over 50–100ms rather than a single step change). This limits the maximum rate of current change (di/dt), preventing the battery voltage from collapsing during initial acceleration.

Verification:

- Re-run the robot through aggressive start-stop and directional reversal test routines.
- Confirm via the multimeter/oscilloscope that the VIN logic rail remains stable above 4.8V during high-torque motor acceleration, and verify that the ESP32 executes continuous runs without a single unexpected reboot.

Task 7 — Wireless Sensor Diagnostics

1. Protocol Selection: Wi-Fi (UDP / AP Mode)

For real-time sensor diagnostics during high-speed robot runs, Wi-Fi (operating in Access Point mode via UDP) is chosen over Classic Bluetooth and BLE:

- Latency & Reliability: BLE suffers from connection interval delays, and Classic Bluetooth adds protocol overhead. Wi-Fi via UDP is connectionless and offers near-instantaneous packet delivery with virtually zero latency. If a single packet is dropped due to a track bump, it doesn't stall the stream.
- Ease of Deployment: By configuring the ESP32 as an Access Point (AP), it broadcasts its own local Wi-Fi network directly on the track. A laptop or phone connects instantly without needing an external router to host a live visualization dashboard.